

Basic Classification Model

K-Nearest Neighbors · Iris Dataset · Python & scikit-learn

93.33%

Model Accuracy

150

Dataset Samples

KNN k=5

Algorithm

80 / 20

Train / Test Split

1. Project Overview

This report documents the complete development and evaluation of a supervised machine learning classification model. The goal was to load a real-world dataset, explore its structure, split it for training and testing, apply a classification algorithm, and rigorously assess performance using standard ML metrics.

The Iris flower dataset was selected for its clean structure, balanced classes, and multi-class nature. The K-Nearest Neighbors (KNN) algorithm was applied for its simplicity, strong interpretability, and solid baseline performance on small datasets.

2. Dataset Overview

The Iris dataset (Fisher, 1936) contains 150 samples of iris flowers, each described by four numeric measurements. The target variable is the flower species — one of three classes, each with exactly 50 samples.

Property	Detail
Total Samples	150
Features	4 — Sepal Length, Sepal Width, Petal Length, Petal Width
Target Classes	3 — Setosa, Versicolor, Virginica
Samples per Class	50 each (perfectly balanced)
Missing Values	None
Sepal Length	4.3 cm – 7.9 cm (mean: 5.84)
Sepal Width	2.0 cm – 4.4 cm (mean: 3.06)
Petal Length	1.0 cm – 6.9 cm (mean: 3.76)
Petal Width	0.1 cm – 2.5 cm (mean: 1.20)

3. Methodology

The project followed a six-step supervised learning pipeline:



Figure 1 — End-to-end ML pipeline

3.1 Train / Test Split

The 150-sample dataset was divided into 80% training (120 samples) and 20% testing (30 samples) using stratified sampling, ensuring each class is proportionally represented in both sets. A fixed random_state=42 guarantees full reproducibility.



Figure 2 — Dataset split visualization

3.2 Feature Scaling

StandardScaler normalizes all features to zero mean and unit variance. This step is critical for KNN — the algorithm uses Euclidean distance, so features on larger scales would dominate the distance calculation without normalization. The scaler was fitted exclusively on training data to prevent data leakage into the test set.

3.3 K-Nearest Neighbors Algorithm

KNN classifies a sample by finding the k most similar training points and assigning the majority class among them. It stores all training data and makes predictions at query time — making it a lazy, non-parametric learner. k=5 was chosen as a balanced starting value that avoids both over- and underfitting.

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train) # fit only on train
X_test_s = scaler.transform(X_test) # transform test only

model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train_s, y_train)
y_pred = model.predict(X_test_s)
```

4. Results & Evaluation

93.33%

Overall Accuracy

28 out of 30 test samples correctly classified

4.1 Confusion Matrix

The confusion matrix shows the count of correct and incorrect predictions for each class. The diagonal represents correct classifications. Only 2 Virginica samples were misclassified as Versicolor — a known overlap due to similar petal dimensions.

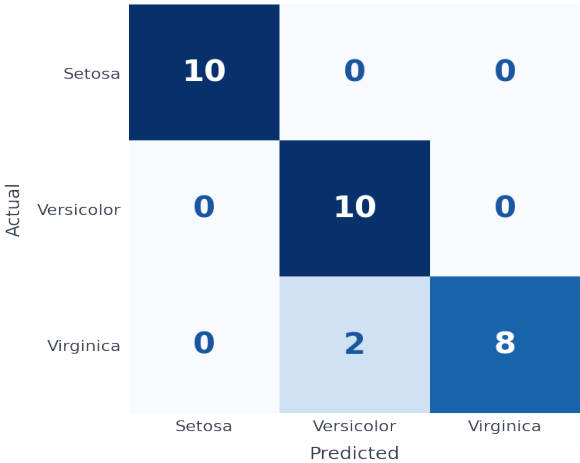


Figure 3 — Confusion matrix (diagonal = correct predictions)

4.2 Per-Class Performance

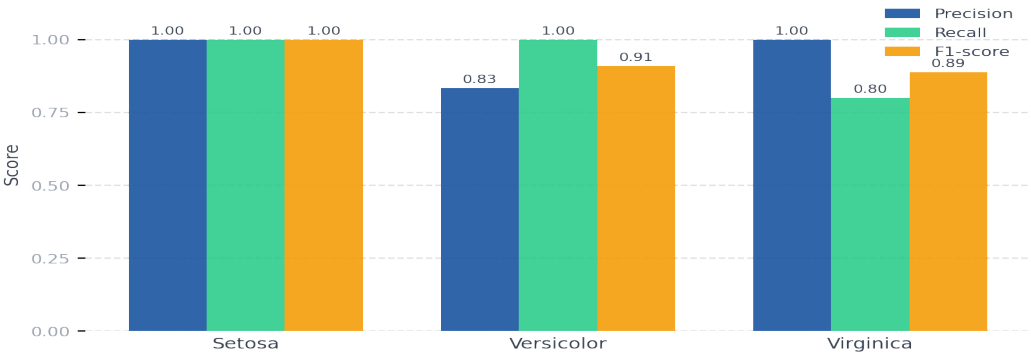


Figure 4 — Precision, Recall, and F1-score per class

Class	Precision	Recall	F1-Score	Support
Setosa	1.00	1.00	1.00	10
Versicolor	0.83	1.00	0.91	10
Virginica	1.00	0.80	0.89	10
Macro Average	0.94	0.93	0.93	30

5. Analysis & Key Learnings

Perfect Setosa Classification	Setosa is linearly separable from the other two classes — KNN captured this boundary with flawless precision and recall.
Virginica vs. Versicolor Overlap	2 Virginica samples were predicted as Versicolor due to overlapping petal length and width distributions between these species.
Importance of Feature Scaling	Without StandardScaler, sepal/petal length (range ~5 cm) would dominate petal width (range ~2 cm), completely distorting distance calculations.

Stratified Splitting	Using stratify=y ensures each class contributes exactly 10 samples to the test set, making evaluation fair and statistically valid.
Preventing Data Leakage	The scaler was fitted only on training data. Fitting on the full dataset would leak test distribution info into training — a critical ML mistake.

6. Conclusion

The KNN classifier achieved 93.33% accuracy on the Iris test set, correctly identifying 28 of 30 samples. Setosa was classified perfectly; only minor confusion occurred between Virginica and Versicolor due to naturally overlapping feature distributions.

This project successfully demonstrated the complete supervised learning workflow — from data loading and exploration, through preprocessing and model training, to rigorous multi-metric evaluation. The skills applied here are directly transferable to real-world ML systems and form the foundation of production-grade machine learning pipelines.

Key Skills Demonstrated	
Data Loading & Exploration	pandas DataFrame, class distribution, descriptive statistics
Train / Test Splitting	Stratified split, random_state, 80/20 ratio
Feature Engineering	StandardScaler — fit vs. transform discipline
Supervised Classification	KNN — distance-based, non-parametric learning
Model Evaluation	Accuracy, Precision, Recall, F1-Score, Confusion Matrix